AI Deep Drawability

July 2026 Project Update

Date: 2026-07-05

Author: Paul Lintzen

1 Updates

1.1 ConvNeXtv2

I began this week trying to address the data imbalance between good and bad classes. As was previously identified, we have 76,848 total bad samples and only 42,000 good ones and our accuracies our model performs meaningfully better on the bad samples. In order to hopefully improve the accuracy on good samples I attempted to use Synthetic Minority Over-sampling Technique (SMOTE) to generate more good samples. This technique attempts to learn the underlying structure of the minority class and produce synthetic samples to balance out class counts. The big limitation of this approach is that if there is no clear separation between the classes and we have heavy overlap SMOTE will just produce more overlapping samples.

Figure 1 shows the baseline results based off the final model from last week. Figure 2 shows our new results once we use SMOTE to generate synthetic samples. We can see that our accuracy decreased for the bad sheets while improving for the good ones. We can also see that SMOTE is working and we now have equal amounts of good and bad samples in each training set. If we average all sheets we went from an overall accuracy of 0.591 to 0.595. While this is an increase it is so small that I would say SMOTE isn't contributing meaningfully to the model. Also, since our main priority is catching bad sheets reliably the hit to bad sheet accuracy isn't worth the small increase to good sheet accuracy in my opinion. Overall, while try

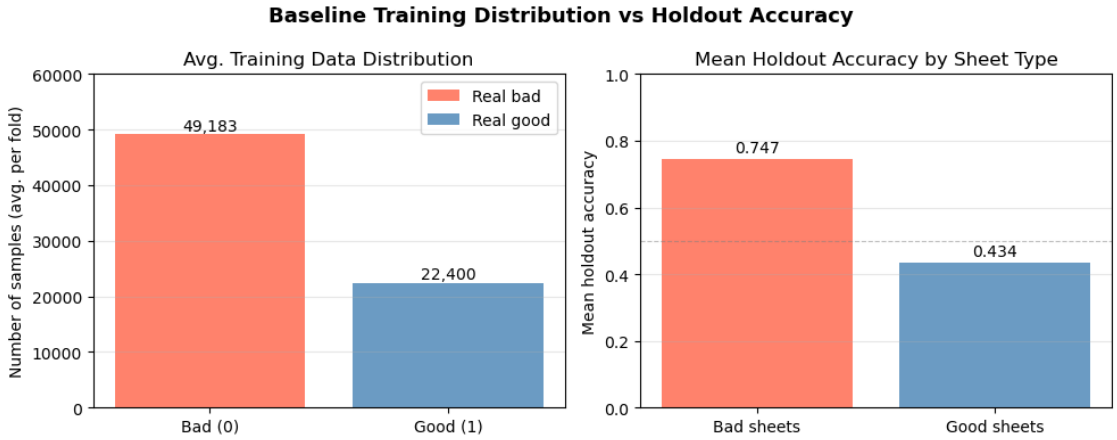

Figure 1: Previous accuracy compared to data distribution

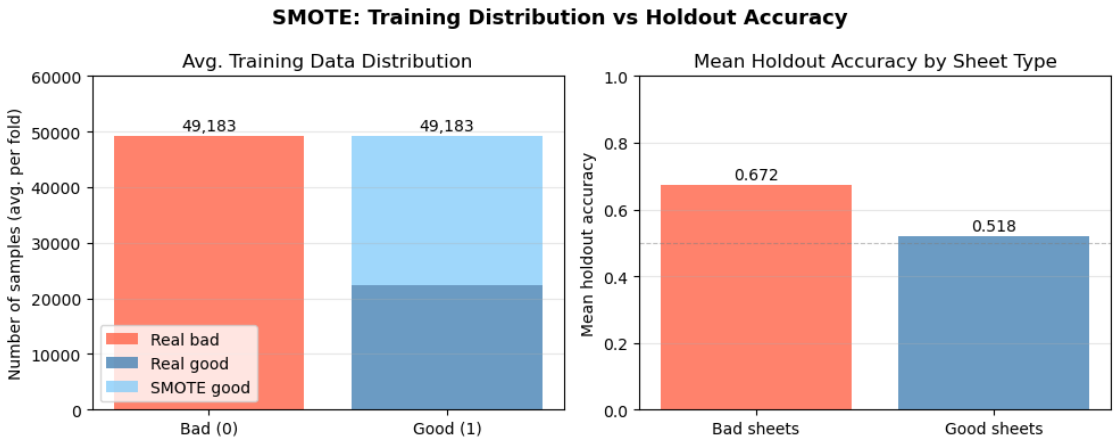

Figure 2: New results using SMOTE

As a slightly different approach to addressing our data imbalance, I tried using weighted cross entropy loss. This means that in training we no longer treat a wrong prediction of any sample equally. Now, incorrect predictions on good samples are weighted proportionately to the count of good vs. bad samples in the current fold (roughly 2x). This means that even with imbalanced data our model is incentivized to predict equally well for both classes. Figure 3 shows the results from using this method. We see an increase in both good and bad accuracy over SMOTE. The average across both classes is now 0.603 which, while still small, is another little step up over our initial model. My takeaway from this is that the overlap between our good and bad samples is high so generating synthetic "good" samples doesn't help our model as much as just weighing the real good samples proportionally. And while we have some imbalance, it's not so large that weighing our loss terms changes the performance drastically. It's more of a fine tuning step.

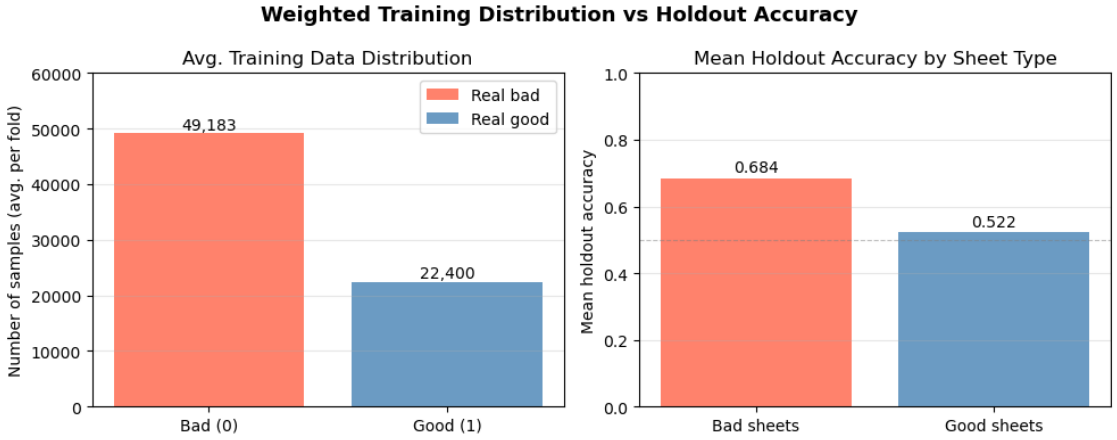

Figure 3: New results using class weights

Overall, I think we are still limited by the relatively small amount of total sheets we have. With only 3 good and 5 bad and a need to hold out 1 of each to get fair representation of performance on unseen sheets, our model is still unable to fully differentiate between sheet to sheet variance and the characteristics that determine if a sheet is good or bad. We perform meaningfully better on bad sheets because each model trains on 4 unique ones while it only gets 2 unique good ones.

1.2 DINOv3

As more data isn't an immediate possiblity I moved to testing some different models. The new models I tested is are newer pretrained models from Meta in the DINOv3 family. Currently we are using ConvNeXtv2 which is a highly optimized Convolutional Neural Network (CNN) whereas DINOv3 is Meta's newest class of Vision Transformer models (ViT). I wanted to see if it would show any performance improvements for our application. As DINOv3 is a gated model I first had to request access. After approval, there 12 provided models to choose from. I started by going with vitb16 which is the basic medium sized model with an embedding dimension of 768. This keeping our embedding dimensions the same as with the current model which makes updating the pipeline simpler. Figure 4 shows the baseline results for the DINOv3-vitb16 model. Compared to our baseline results with ConvNeXtV2 we have slightly lower bad accuracy but higher good accuracy. Our overall accuracy goes from 0.591 to 0.602. This is an improvement but similar to our attempts with adjusting the class balance its relatively small.

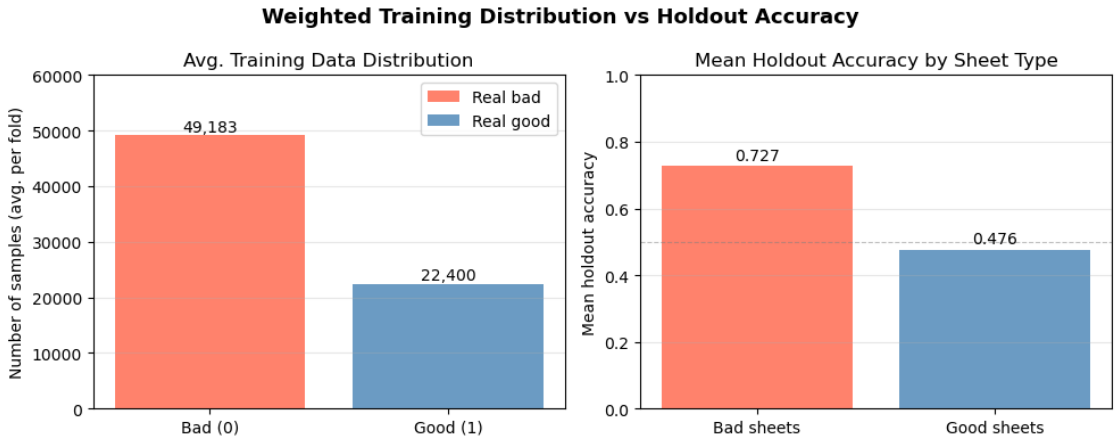

Figure 4: Baseline medium DINOv3 Model (NOTE: Title is wrong - not weighted)

Next I tried using the same class balancing method from earlier to see if I could improve performance slightly more. Figure 5 shows the results. This is our best result yet with an overall accuracy of 0.611.

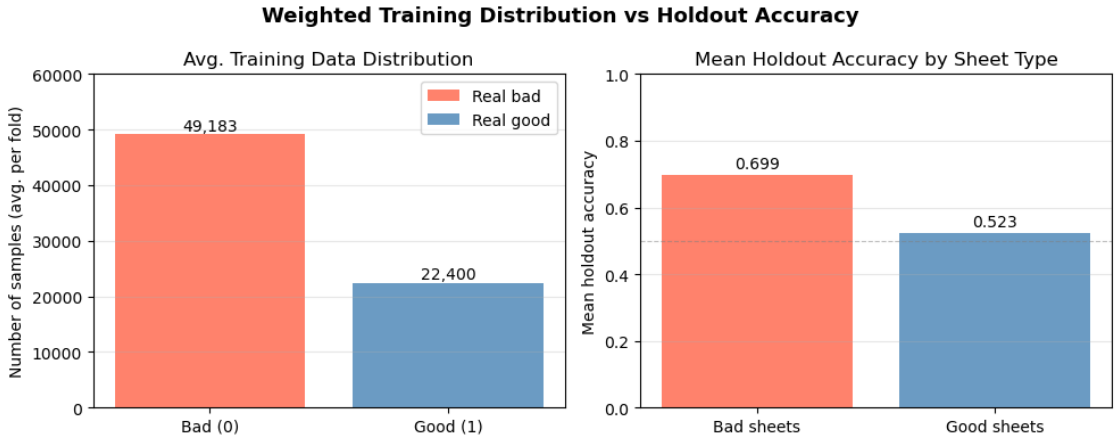

Figure 5: Weighted medium DINOv3 Model

As an additional exploration I tried using the large DINOv3-vitl16 model and the large DINOv3

model with ConvNeXt backbone. The have 300M and 186M parameters respectively versus 86M for the medium DINOv3 and 28M for ConvNeXtv2. These larger models took longer to train and had embedding dimensions of 1024 and 1536 instead of the current 768. But, as we can see in Figure 6, this additional complexity alone did not net meaningful improvements.

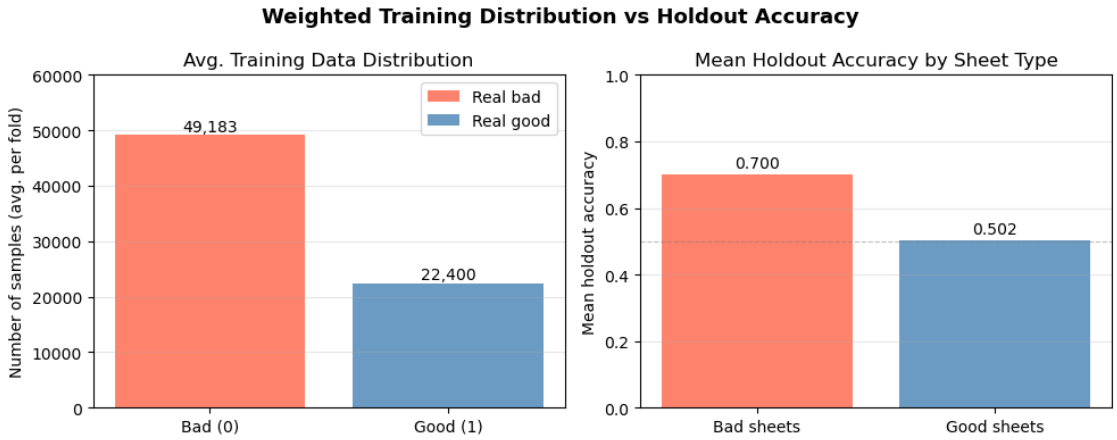

Figure 6: Weighted large DINOv3 Model

Looking at the ConvNeXt version result in Figure 7, we can see a drop in bad sheet accuracy but increase in good sheet to get an overall accuracy of 0.604 which is slightly better than the large ViT only model but again only by a tiny margin.

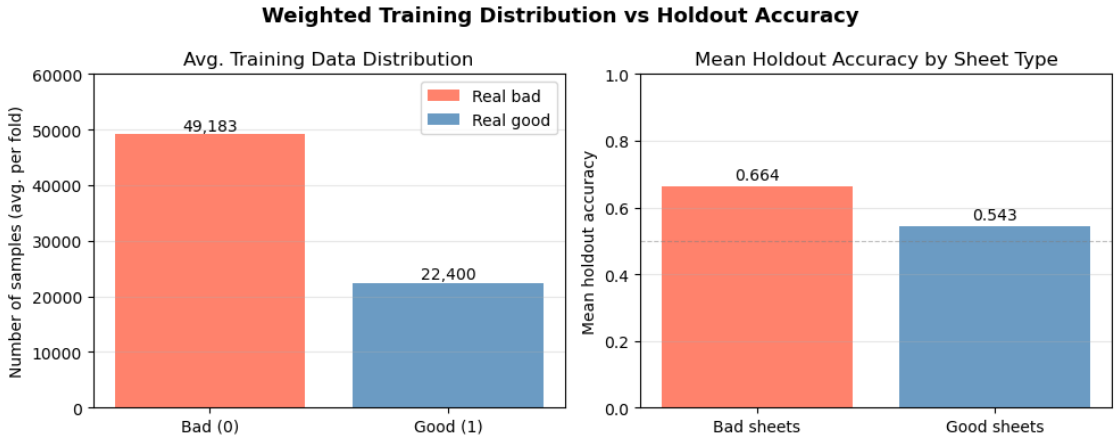

Figure 7: Weighted Large DINOv3 ConvNeXt Model

This model seemed promising so I also tested it without the weighted classes and got the results in Figure 8. As expected this resulted in better accuracy on the bad sheets and worse on the good sheets. The overall stayed the same at 0.604.

One interesting thing with this new model worth highlighting is that for the first time we have some correct predictions for sheet 2123. Figure 9 shows the precision and recall results. We're still predicting wrong for 99% of the samples from this sheet but it's an improvement over the previous 100%.

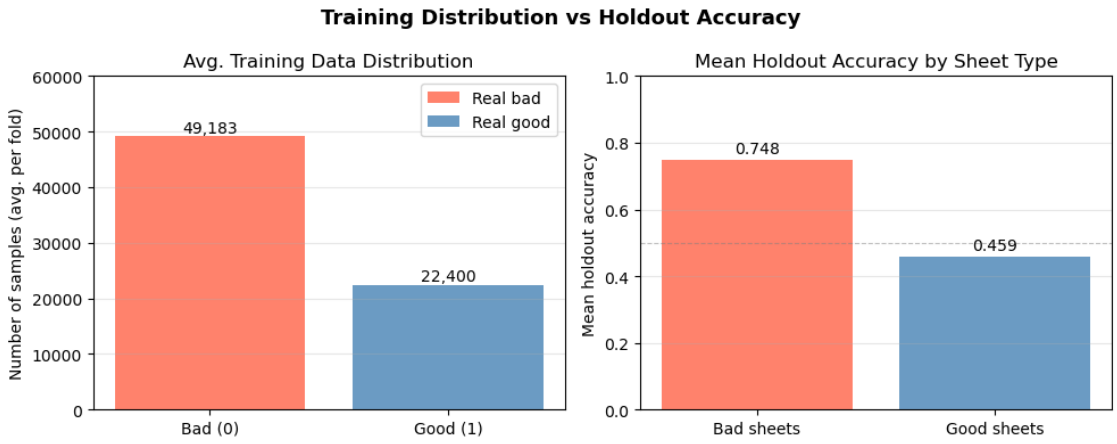

Figure 8: Unweighted Large DINOv3 ConvNeXt Model

1.3 Conclusion

To get a final out-of-fold test accuracy for each of our attempted models I tested each sheet on the models where it was heldout and averaged the accuracies. All of the models sat between 0.60 and 0.62 so there was no clear winner. If I had to chose one the DINOv3 ConvNeXt model seems to have a slight edge but the margin is so small it may just be down to random noise.

Overall, this exploration has shown me that the underlying shortcomings in our data can't be fixed by model choice or synthesizing new data. We can fine tune and get some small performance gains or adjust the balance between accuracy on good vs bad sheets but overall we seem to be hitting a limit.

2 Next Steps

Given that our test accuracy seems to be stuck around 0.62, I believe this is approaching the limit of what is possible with the data we currently have. With only 8 unique sheets our model is still getting stuck on some sheet to sheet variations and failing to fully learn what separates good vs. bad. Next, I will build a per sheet classifier model to possibly leverage those sheet-level differences to improve predictions. Additionally, I will continue experimenting with different approaches including contrastive learning. I will also consult with Dr. Pak to see his interpretation of my results with SMOTE and the different models I tried.

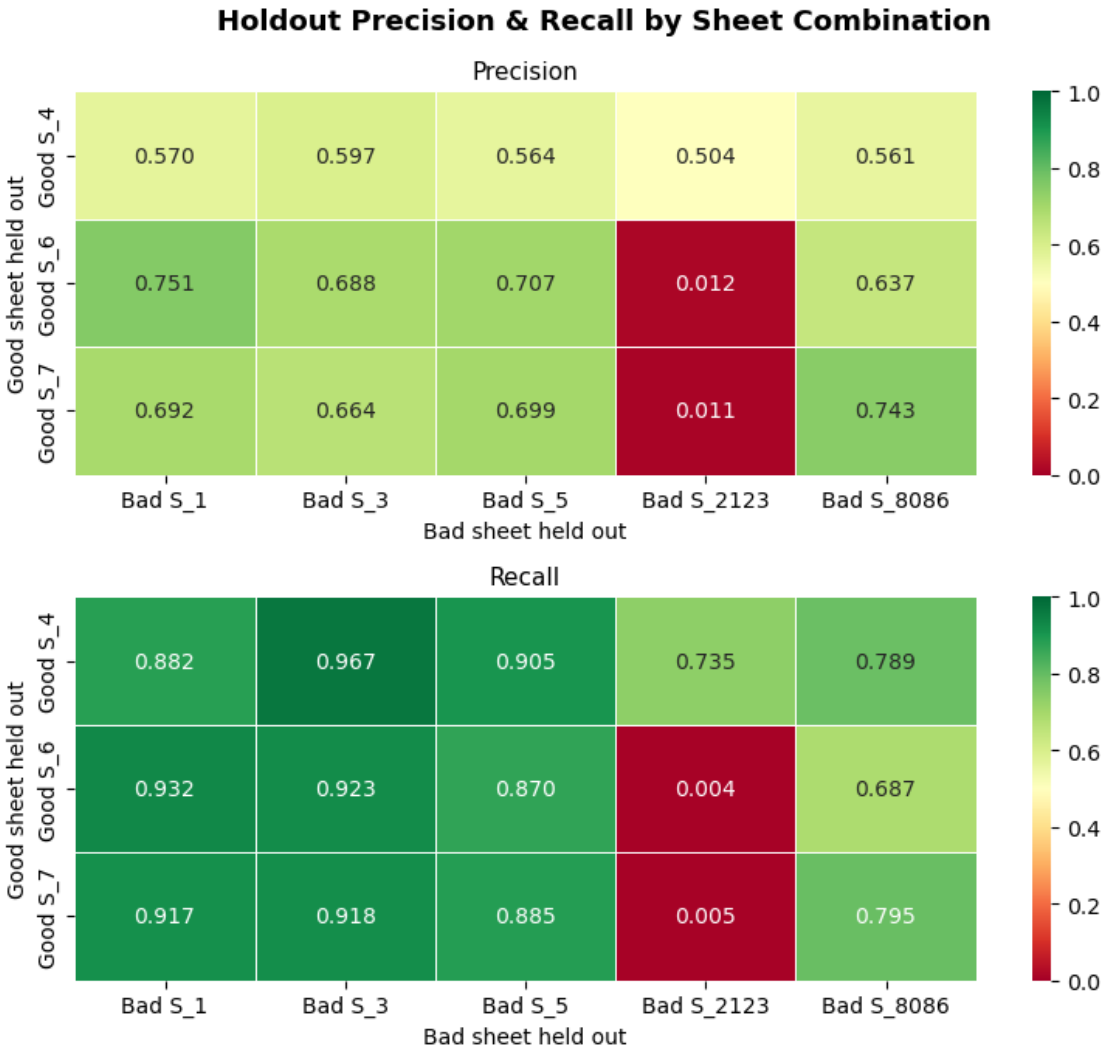

Figure 9: Unweighted large DINOv3 ConvNeXt model precision and recall